BÀI THỰC HÀNH 1

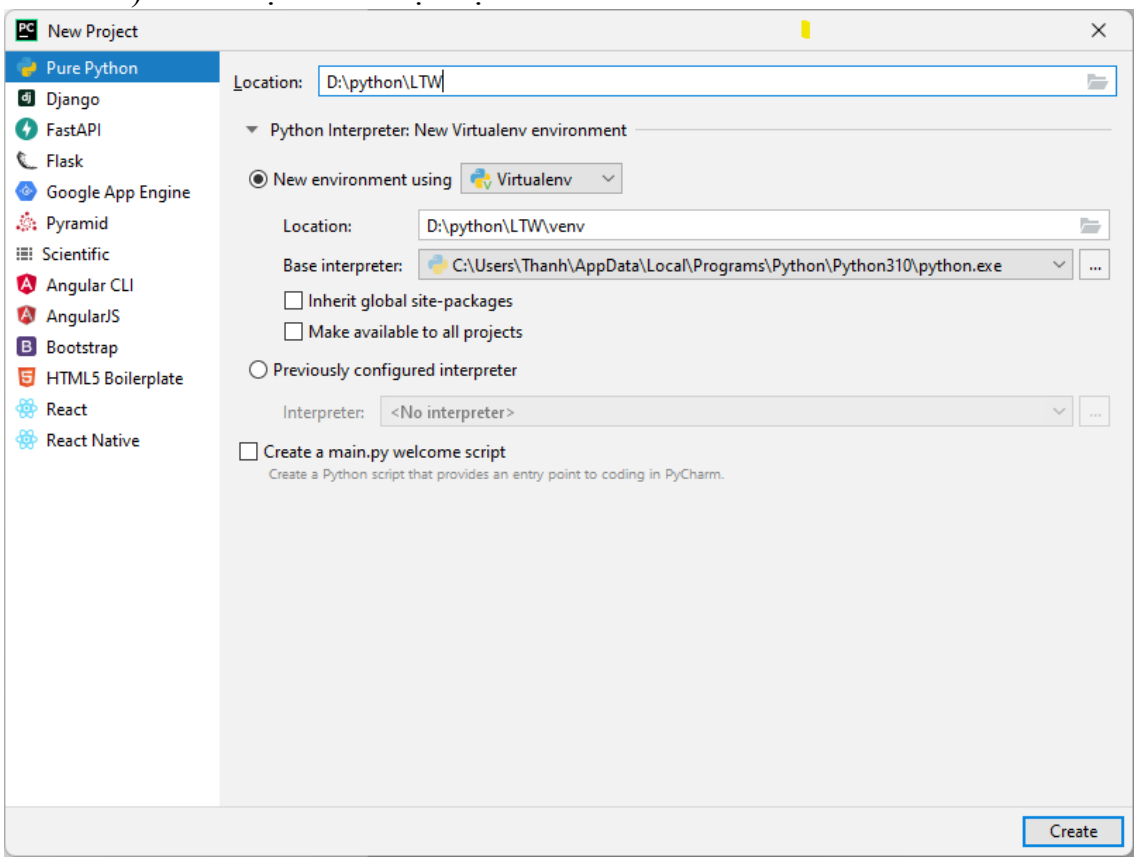
Xuyên suốt các bài thực hành trong học phần này, chúng ta sẽ tiến hành xây dựng một ứng dụng web phục vụ đánh giá sách tên là Bookr. Qua từng chương, tùy theo nội dung học, các tính năng sẽ lần lượt được bổ sung vào ứng dụng này.

Trong chương 1, chúng ta sẽ thực hành các bước đầu tiên để tải về và cài đặt Django, tạo dự án, thực hiện các thao tác thiết đặt ban đầu và tạo App đơn giản sử dụng Model, View và Template.

I. Cài đặt PyCharm và Django

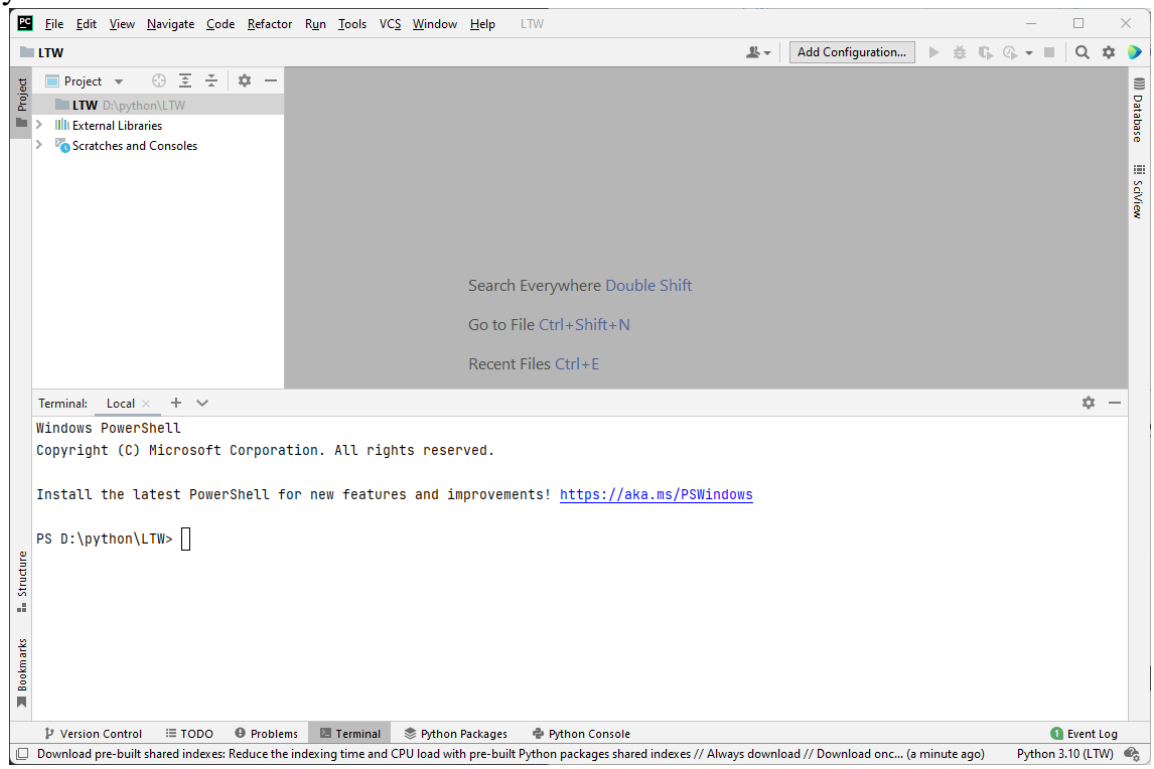
A. Cài đặt PyCharm

1. Truy cập vào trang **Download** trên website của Hãng JetBrains tại đường link sau: https://www.jetbrains.com/pycharm/download
2. Chọn tải về phiên bản **PyCharm Community Edition**
3. Tiến hành cài đặt theo hướng dẫn.
   Lưu ý: *Nên cài đặt Python trước khi tiến hành cài đặt PyCharm.*
4. Sau khi cài đặt hoàn tất. Mở PyCharm và chọn menu File → New Project để tạo thử một dự án Pure Python. Đặt tên dự án là LTW và lựa chọn sử dụng môi trường Python ảo (Virtualenv). Giao diện cửa sổ tạo dự án như hình sau:



Hình I-1. Giao diện cửa sổ New Project để nhập thông tin tạo mới một dự án trên PyCharm

5. Nhấn nút Create để tạo mới dự án. Sau khi hoàn tất tạo mới dự án, cửa sổ Pycharm mở ra có dạng như Hình I-2. Có thể mở bảng terminal để nhập các lệnh Windows PowerShell bằng cách nhấn vào nút **Terminal** ở cạnh dưới hoặc phía bên dưới của cạnh trái cửa sổ PyCharm.



Hình I-2. Giao diện màn hình làm việc của PyCharm

B. Tải về và cài đặt Django

1. Cách 1: Tải về bản 4.2.8 sử dụng **pip**:

```
pip install Django==4.2.8
```

2. Cách 2: Tải về bản phát triển mới nhất trên GitHub với lệnh

```
git clone https://github.com/django/django.git
```

II. Tạo một dự án Django bằng lệnh

Trong bài thực hành này chúng ta sẽ khởi tạo dự án Bookr, chạy trình chủ phục vụ phát triển ứng dụng của Django và tạo App Review bằng cách sử dụng các lệnh.

Yêu cầu: *Hệ thống đã có một môi trường ảo được cài đặt sẵn.*

Tiến hành tạo dự án với các bước sau:

1. Mở terminal và chạy lệnh:

   ```
   django-admin startproject bookr
   ```

   Lệnh này không trả về kết quả nào nhưng sẽ tạo ra một thư mục tên là **bookr** trong thư mục thực thi lệnh. Cấu trúc thư mục như được giới thiệu trong bài học.

Bây giờ chúng ta có thể kiểm tra xem dự án và Django đã được thiết lập chính xác hay chưa bằng cách khởi chạy trình chủ phát triển dự án Django. Việc này được thực hiện với lệnh từ tập tin manage.py như sau:

2. Trong terminal, chuyển đến thư mục **bookr** (sử dụng lệnh CD), và chạy lệnh **manage.py runserver** như sau:

   ```
   python manage.py runserver
   ```

   Lệnh này sẽ khởi chạy trình chủ phát triển của Django. Cửa sổ lệnh xuất hiện kết quả như sau:

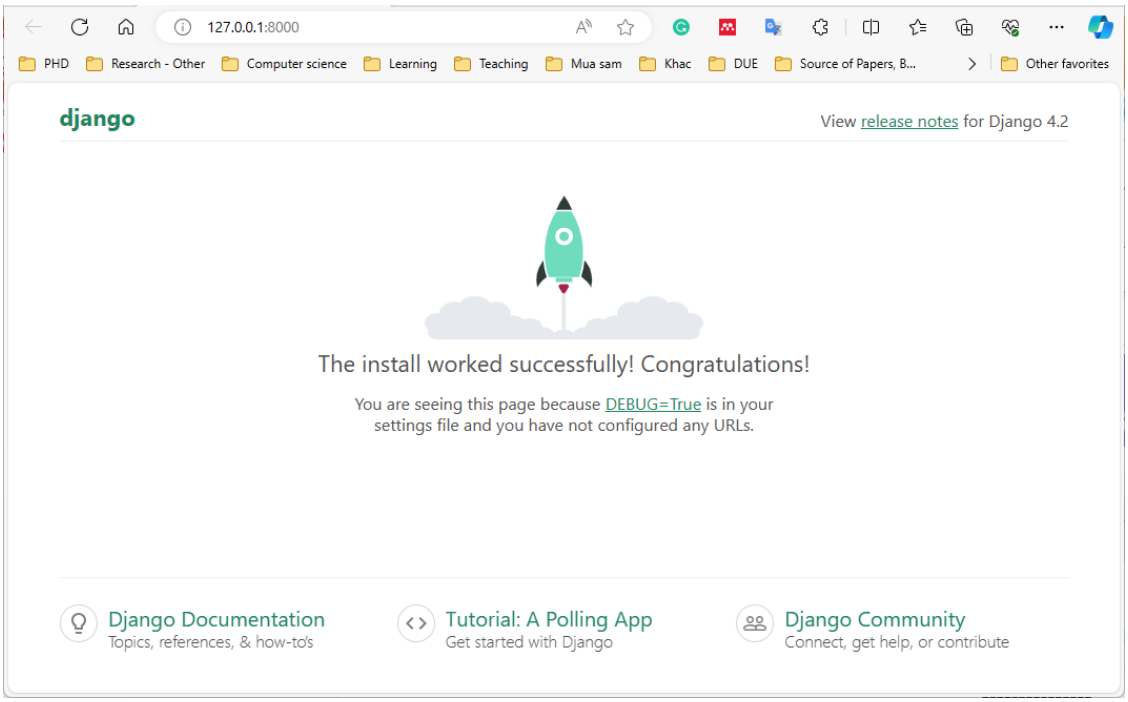
   ```
   Watching for file changes with StatReloader
   Performing system checks...

   System check identified no issues (0 silenced).

   You have 18 unapplied migration(s). Your project may not
   work properly until you apply the migrations for app(s):
   admin, auth, contenttypes, sessions.
   Run 'python manage.py migrate' to apply them.
   January 01, 2024 - 00:00:00
   Django version 4.2.8, using settings 'bookr.settings'
   Starting development server at http://127.0.0.1:8000/
   Quit the server with CTRL-BREAK.
   ```

   Có một số cảnh báo về một số migration nhưng không cần quan tâm ở thời điểm này.

3. Mở trình duyệt web và truy cập đến địa chỉ http://127.0.0.1:8000/ sẽ xuất hiện giao diện màn hình chào mừng của Django như hình sau. Nếu kết quả như hình này, việc thiết lập dự án trên Django đã thành công.

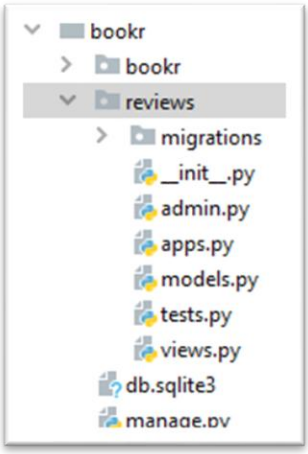


Hình II-1. Giao diện trang web chào mừng của Django

4. Trở lại cửa sổ terminal và dừng trình chủ web bằng tổ hợp phím CTRL-BREAK.
5. Chúng ta sẽ tiếp tục tạo app **reviews** cho dự án này. Trong cửa sổ terminal, tại thư mục dự án **bookr**, thực thi lệnh sau:

```
python manage.py startapp reviews
```

Lệnh này không trả về kết quả trên terminal, như sẽ tạo ra thư mục **reviews** và cấu trúc thư mục dự án **bookr** sẽ như Hình II-2.



Hình II-2. Cấu thúc thư mục dự án Bookr và app Reviews

Chi tiết về cấu trúc thư mục app này đã được giới thiệu trong slide bài học.

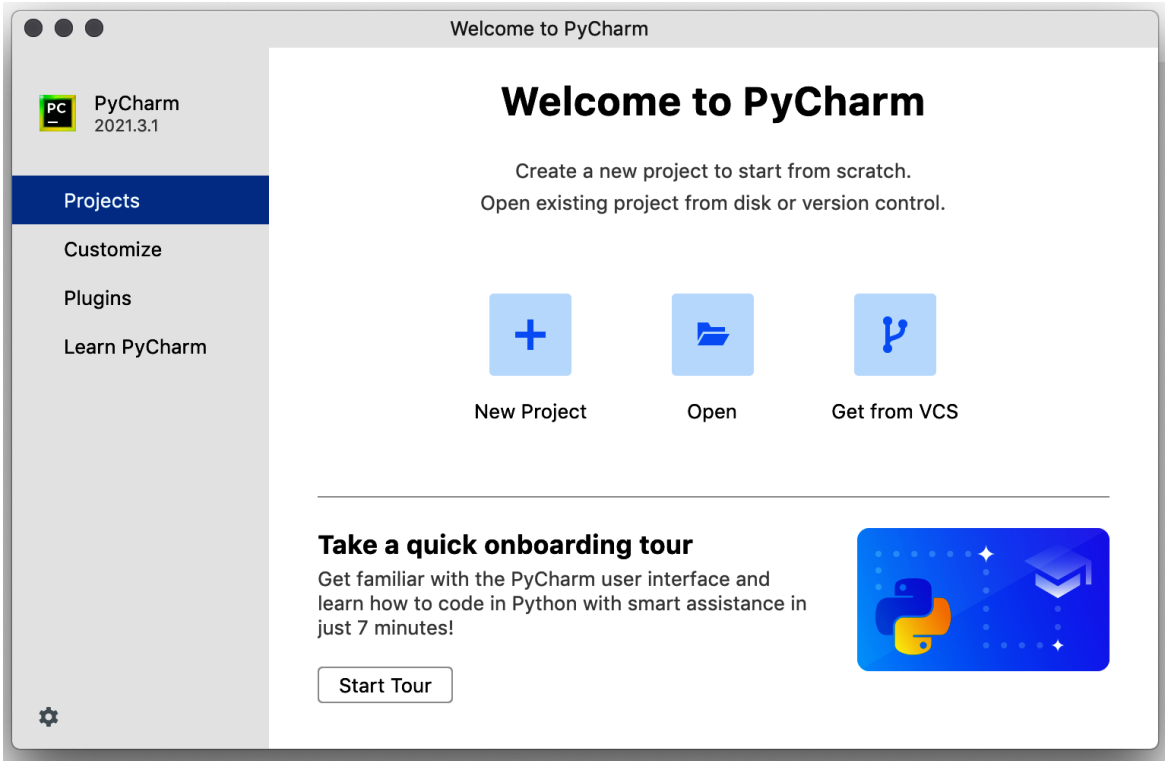
III. Thực hiện một số cấu hình cho PyCharm để chạy dự án Django

A. Thiết đặt để mở một dự án Django đã tạo bên ngoài trên PyCharm

Lưu ý: *Nếu dự án đã tạo trên PyCharm thì không cần thực hiện bài này.*

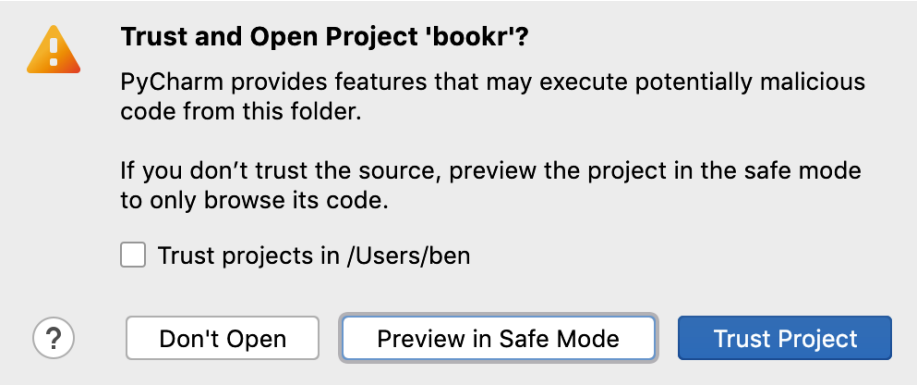
Phần này sẽ thực hành làm việc với dự án Bookr đã tạo trong Phần II trên PyCharm.

1. Mở PyCharm, giao diện chào mừng của PyCharm sẽ có dạng như Hình III-1.



Hình III-1. Màn hình chào mừng của PyCharm

2. Bấm nút Open và chọn đến thư mục dự án bookr đã tạo ở trên để mở. Lưu ý chọn thư mục gốc của bookr, không phải là thư mục bookr con ở bên trong.
3. Để đảm bảo an toàn, PyCharm sẽ xác nhận là dự án này có đáng tin không. Giao diện hộp thoại xác nhận như Hình III-2.



Hình III-2. Hộp thoại xác nhận dự án tin cậy

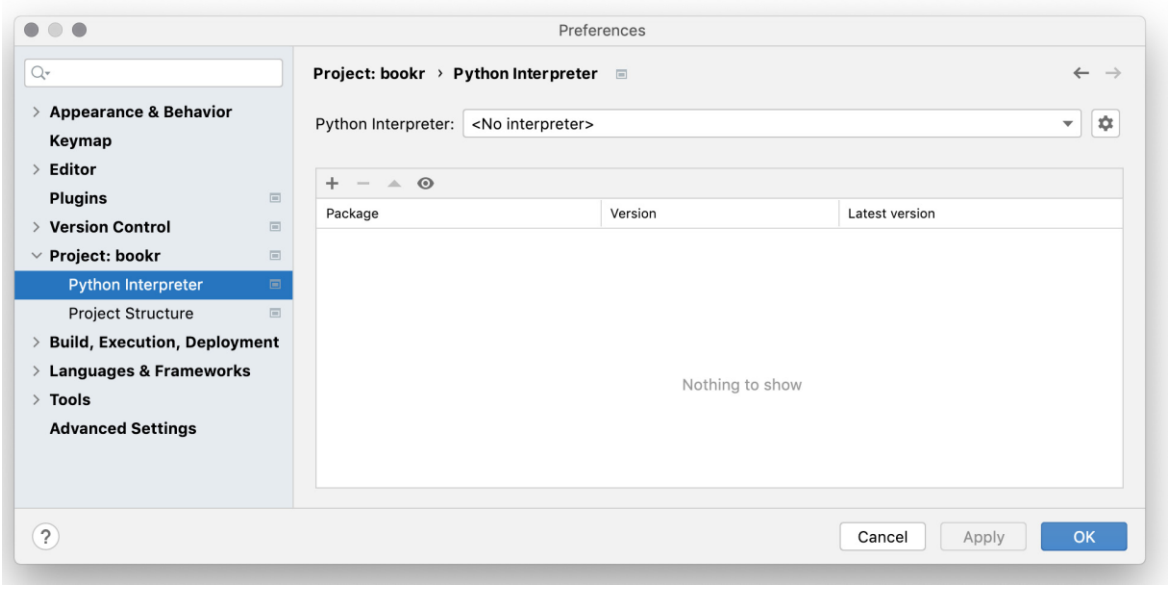
Ngoài ra, PyCharm có thể sẽ yêu cầu chọn theme làm việc phù hợp nếu đây là lần đầu tiên mở ứng dụng này.

Sau khi trả lời xong các câu hỏi, trong khung Project của PyCharn sẽ hiển thị cấu trúc thư mục như ***Hình II-2***.

Để chạy dự án này trên trình chủ phát triển của Django, cần phải thiết đặt bộ thông dịch (interpreter) trong môi trường ảo. Trước tiên cần thêm bộ thông dịch vào phần các thiết đặt trình thông dịch toàn cục theo các bước sau:

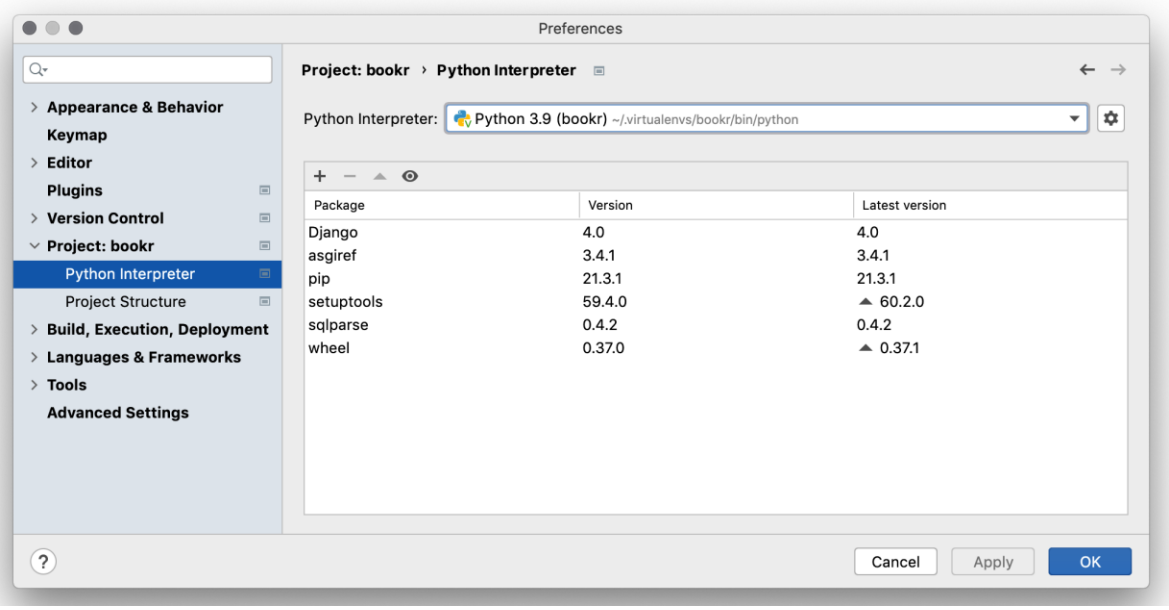
4. Mở cửa sổ Settings (trên Windows/ Linux) hoặc Preferences (trên macOS):
   + Trên macOS: chọn menu PyCharm → Preferences
   + Trên Windows/ Linux: chọn menu File → Settings

5. Trong cửa sổ Settings/ Preferences, ở khung bên trái, chọn Project: bookr rồi chọn Project Interpreter như hình sau:



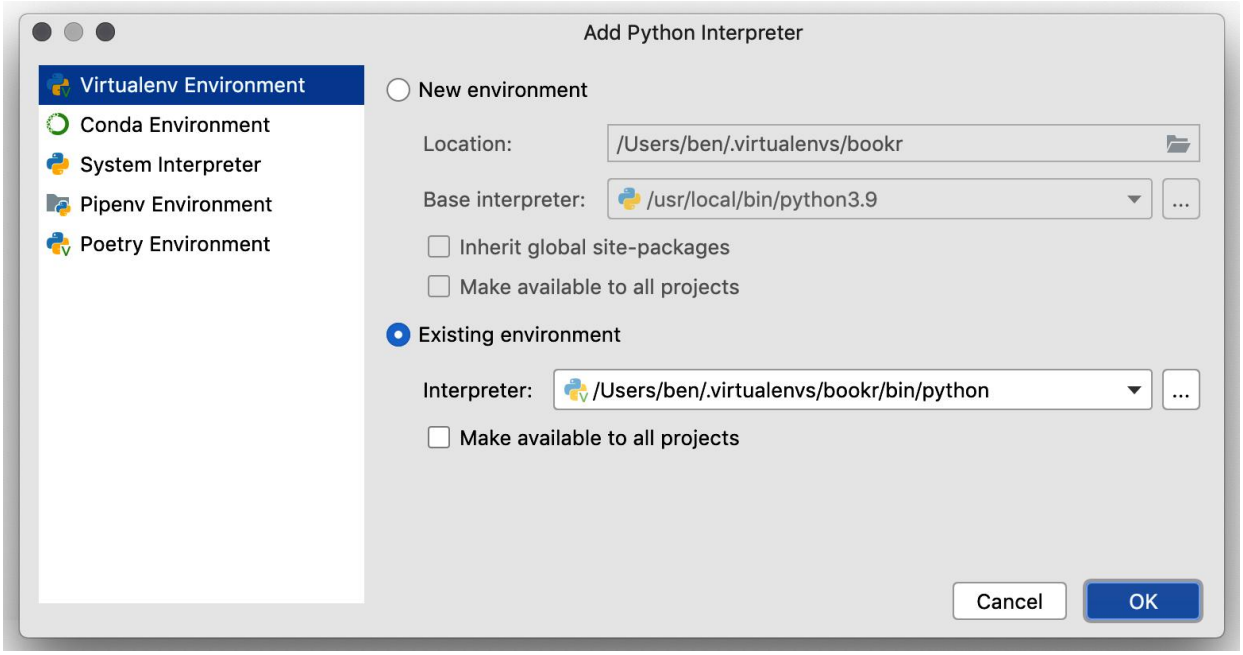
Hình III-3. Cửa sổ Settings/ Preferences của PyCharm

6. Trong một số trường hợp, PyCharm có thể tự động xác định các môi trường ảo. Khi đó cửa sổ này sẽ liệt kê sẵn trình thông dịch của dự án như Hình III-4.



Hình III-4. Trình thông dịch của dự án Bookr

7. Trong hầu hết các trường hợp, bộ thông dịch Python cần được thiết đặt thủ công. Để thực hiện, bấm chuột vào biểu tượng bánh răng phía sau mục Python Interpreter và chọn **Add…**
8. Cửa sổ Add Python Interpreter sẽ xuất hiện như trong Hình III-5. Chọn mục Existing environment và bấm vào nút (…) nằm cạnh mục Interpreter. Sau đó chọn trình thông dịch Python cho môi trường ảo.



Hình III-5. Cửa sổ Add Python Interpreter

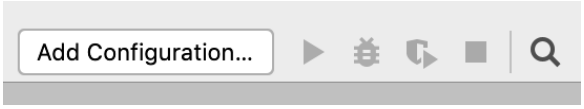
9. Đường dẫn đến trình thông dịch thường ở dạng:
   + Trên Windows: `<thư mục dự án>\venv\scripts\python.exe`
   + Trên macOS: `/Users/<tên người dùng>/.virtualenvs/bookr/bin/python3`
   + Trên Linux: `/home/<tên người dùng>/.virtualenvs/bookr/bin/python3`

10. Sau khi nhấn **OK** để đóng cửa sổ **Add Python Interpreter**, cửa sổ **Settings/ Preferences** sẽ giống như trong Hình III-4
11. Nhấn nút **OK** trên cửa sổ **Settings/ Preferences** để đóng lại. PyCharm sẽ thực hiện lập chỉ mục cho môi trường ảo và các thư viện đã cài đặt. Tiến trình này hiển thị ở dưới thanh trạng thái.

B. Cấu hình nút chạy trình chủ phát triển Django trên PyCharm

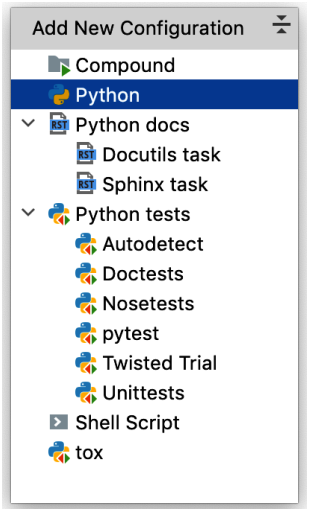
Để thực hiện chạy trình chủ phát triển Django, Python cần được cấu hình với một bản cấu hình thực thi như sau:

1. Nhấn nút **Add Configuration…** nằm trên thanh công cụ ở phía trên bên phải của cửa sổ dự án PyCharm để mở cửa sổ **Run/ Debug Configurations**.



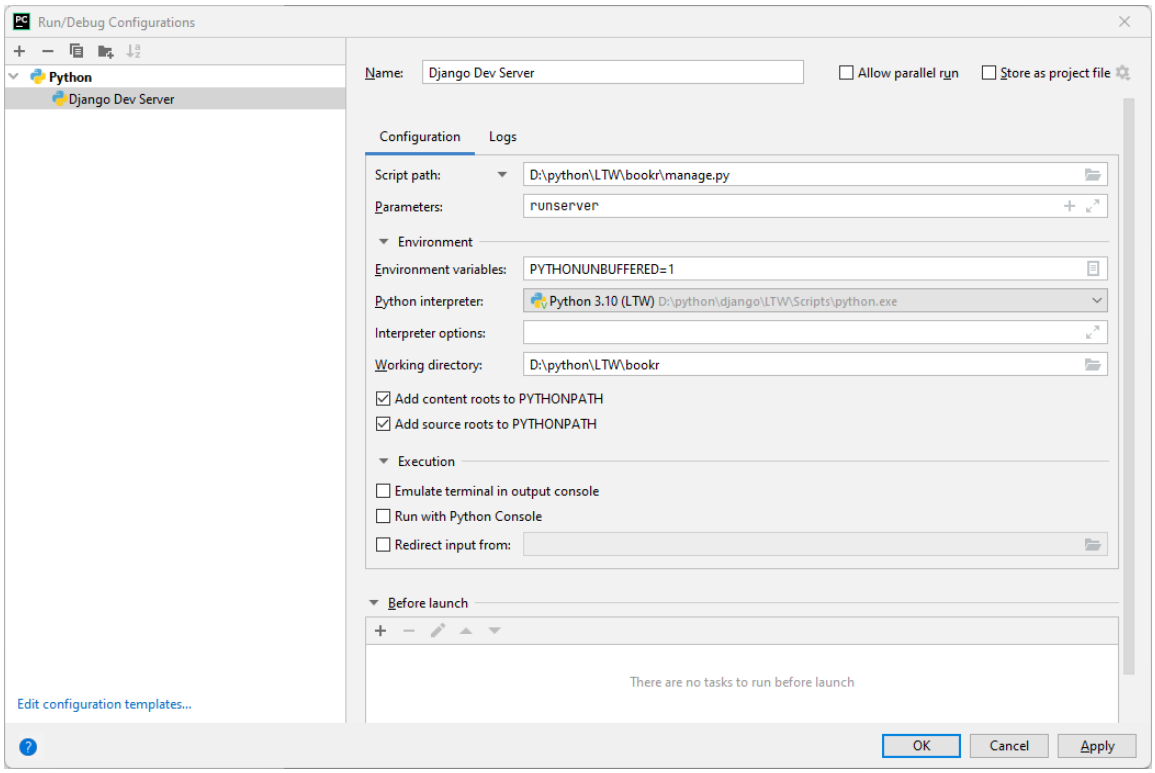
*Hình III-6. Vị trí nút **Add Configuration…** trên thanh công cụ*

2. Bấm vào nút dấu cộng (+) nằm phía trên bên trái cửa sổ và chọn Python trong menu sổ xuống.



*Hình III-7. Khối Add New Configuration trong cửa sổ **Run/ Debug Configurations***

3. Một khung thêm mới bản cấu hình xuất hiện bên phải cửa sổ **Run/ Debug Configurations** như Hình III-8.



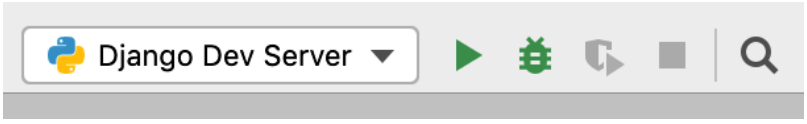
*Hình III-8. Cửa sổ **Run/ Debug Configurations***

Tiến hành điền thông tin vào các mục như sau:

+ **Name**: Tên của bản cấu hình, nhập vào `Django Dev Server`
+ **Script path**: đường dẫn đến tập tin manage.py. Nếu bấm vào biểu tượng thư mục ở mục này thì có thể duyệt đến tập tin manage.py trong thư mục dự án bookr.
+ **Parameters**: các tham số truyền vào khi chạy lệnh manage.py (giống như trong dòng lệnh ở trên). Sử dụng tham số giống như khi khởi động trình chủ là: `runserver`

4. Mục **Python interpreter** sẽ được đặt tự động. Đây cần là mục có tên như đã đặt ở phần **Add Python Interpreter** ở trên. Nếu không đúng thì cần chọn lại.
   Mục **Working directory** là thư mục của dự án Bookr.

5. Đánh dấu chọn cho mục **Add content roots to PYTHONPATH** và **Add source roots to PYTHONPATH**. Tùy chọn này đảm bảo rằng PyCharm sẽ thêm thư mục dự án Bookr vào đường dẫn PYTHONPATH. Nếu khong chọn mục này, các mục nhập từ dự án sẽ không hoạt động đúng.
6. Kiểm tra nội dung cửa sổ **Run/ Debug Configurations** đảm bảo tương tự Hình III-8 rồi nhấn nút OK để lưu bản cấu hình
7. Bây giờ có thể khởi chạy trình chủ phát triển Django bằng nút **Play** trên thanh công cụ thay vì phải nhập lệnh trong cửa sổ terminal.



Hình III-9. Nút Play, Debug và Stop trên thanh công cụ dự án

8. Sau khi nhấn nút Play, kết quả hiển thị ở phần Run sẽ tương tự như hình sau.



Hình III-10. Kết quả hiển thị ở phần Run của cửa sổ dự án

9. Mở trình duyệt và truy cập vào địa chỉ http://127.0.0.1:8000. Kết quả sẽ tương tự ở Phần II.

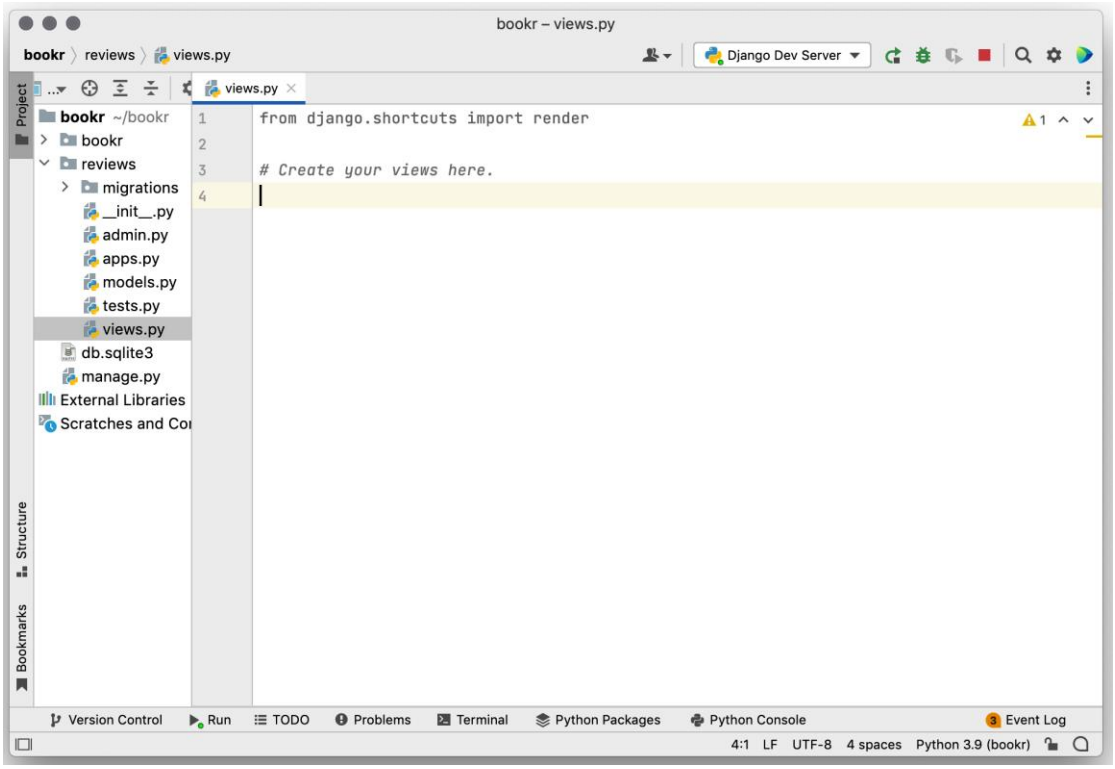
IV. Tạo và thiết lập truy cập View

A. Tạo View hiển thị nội dung tĩnh

Bài này sẽ tạo một View đơn giản để hiển thị nội dung văn bản tĩnh.

1. Trong PyCharm, mở rộng thư mục **reviews** trong khung **Project**, sau đó bấm đôi vào tập tin **views.py** để mở. Trong khung biên tập tập tin ở bên phải, nội dung tập tin sẽ như sau:

```
from django.shortcuts import render
# Create your views here.
```



Hình IV-1. Nội dung mặc định trong tập tin views.py

2. Nhập vào đoạn mã như sau:

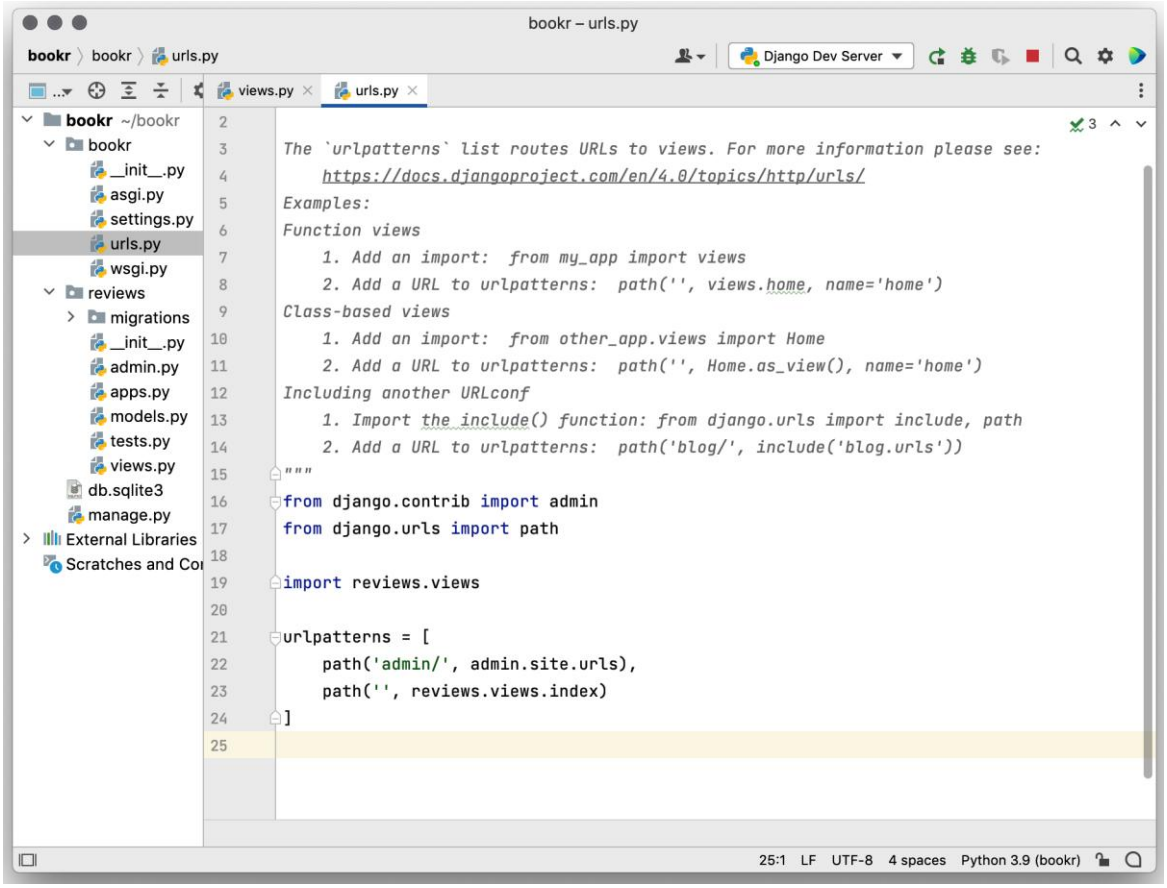
```
from django.http import HttpResponse
def index(request):
    return HttpResponse("Hello, world!")
```

Đầu tiên cần import lớp `HttpResponse` từ `django.http`. Lớp này dùng để tạo phản hồi đến trình duyệt web cũng như kiểm soát các tham số khác như HTTP header hay mã trạng thái. Trong bài này, chọn header mặc định và mã trạng thái `200 Success`. Tham số đầu tiên của `HttpResponse` là nội dung của thông điệp phản hồi, ở đây là "Hello, world!".

3. Thiết lập ánh xạ URL đến view index bằng cách thêm đoạn mã sau vào tập tin urls.py

```
import reviews.views #import view reviews

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', reviews.views.index)
]
```



Hình IV-2. Nội dung tập tin urls.py sau khi chỉnh sửa

4. Chuyển trở lại trình duyệt web và load lại trang http://127.0.0.1:8000, kết quả sẽ như hình sau:



Hình IV-3. Giao diện trang web với view index

B. Tạo View sử dụng dữ liệu từ GET Request

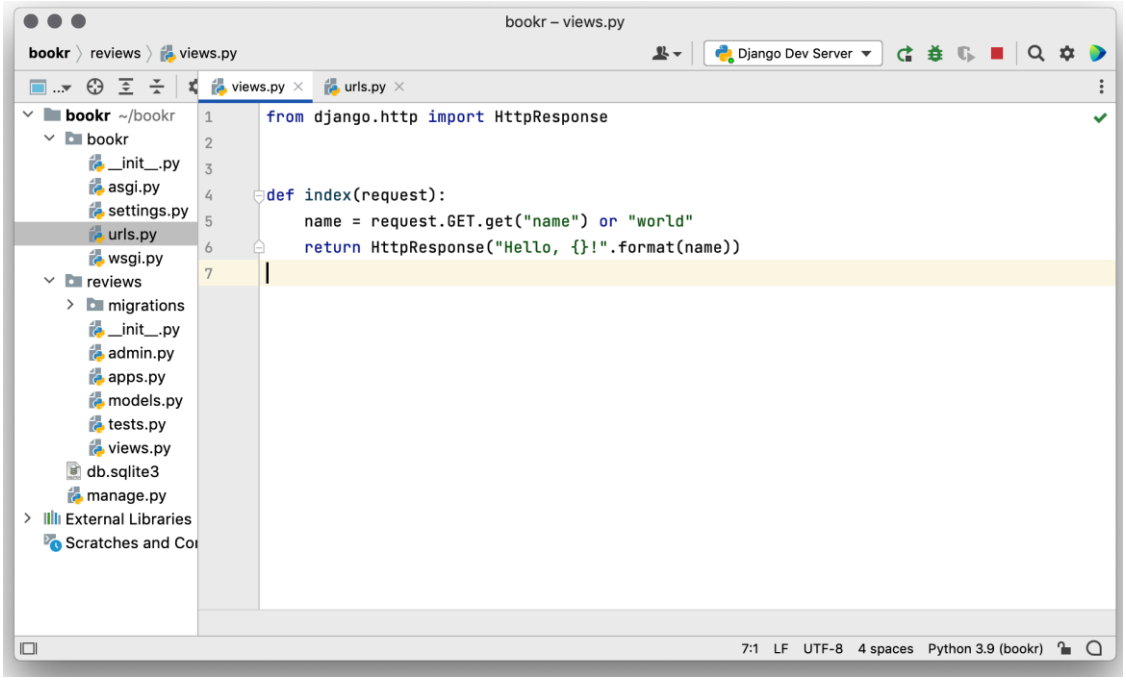
Phần này sẽ tạo một View nhận tên một người từ HTTP GET Request và hiển thị thời chào đến người đó.

1. Mở tập tin views.py trên PyCharm. Theo một biến mới tên là `name` lấy dữ liệu từ GET Request. Thêm dòng sau ngay dưới dòng định nghĩa hàm `index`.

```
name = request.GET.get("name") or "world"
```

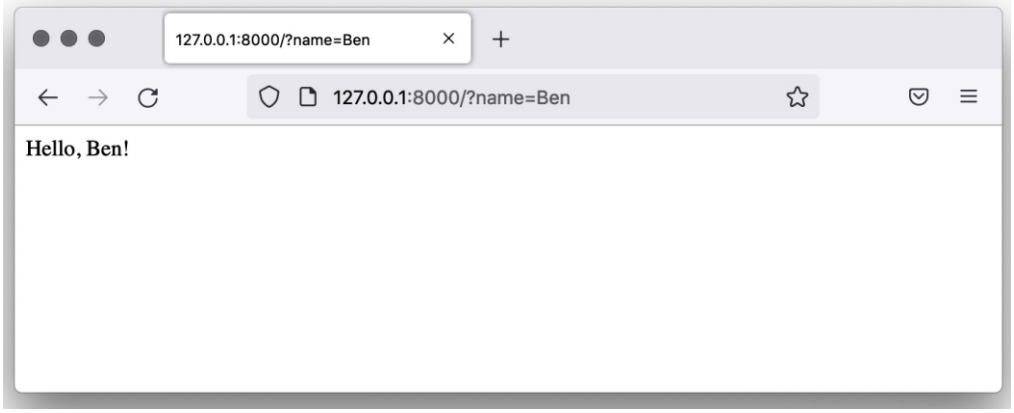
2. Thay đổi giá trị trả về của hàm `index` để một phần của văn bản hiển thị sử dụng giá trị của biến `name` như sau:

```
return HttpResponse("Hello, {}!".format(name))
```



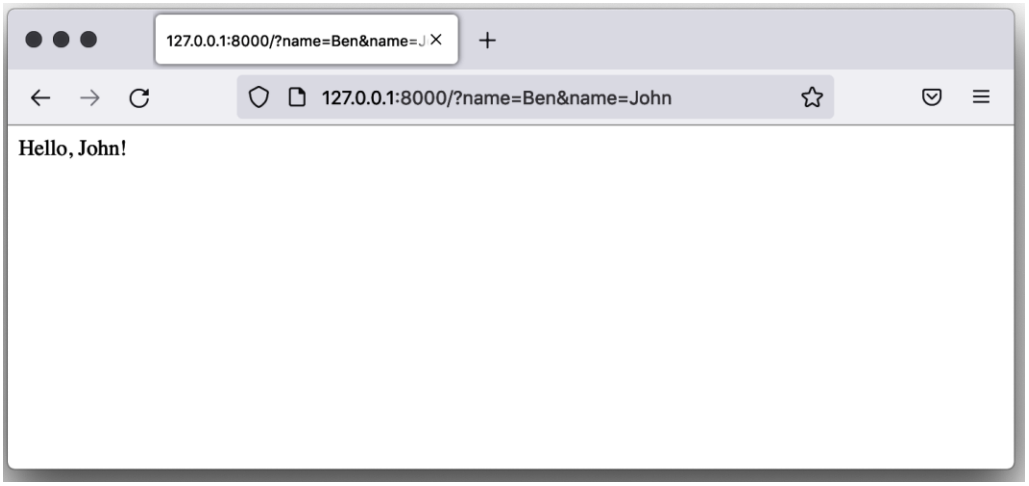
Hình IV-4. Nội dung hàm index sau khi điều chỉnh

3. Mở trình duyệt Web và truy cập vào địa chỉ http://127.0.0.1:8000. Kết quả trả về sẽ vẫn là **Hello, world!** do chúng ta chưa truyền tham số cho thông điệp GET trên URL. Thêm tên một người vào URL, ví dụ http://127.0.0.1:8000?name=Ben, kết quả sẽ như Hình IV-5.



Hình IV-5. Kết quả trang web khi truyền tham số qua URL

4. Thử thêm 2 tên, ví dụ http://127.0.0.1:8000?name=Ben&name=John. Kết quả là chỉ có giá trị cuối cùng được sử dụng và trang web sẽ hiển thị là **Hello, John!**



Hình IV-6. Kết quả trang web với nhiều giá trị name được gán qua URL

5. Thử truyền giá trị name là trống như sau: http://127.0.0.1:8000?name=. Trang web sẽ hiển thị lại là Hello, world!



Hình IV-7. Kết quả trang web với giá trị name để trống

V. Sử dụng Template

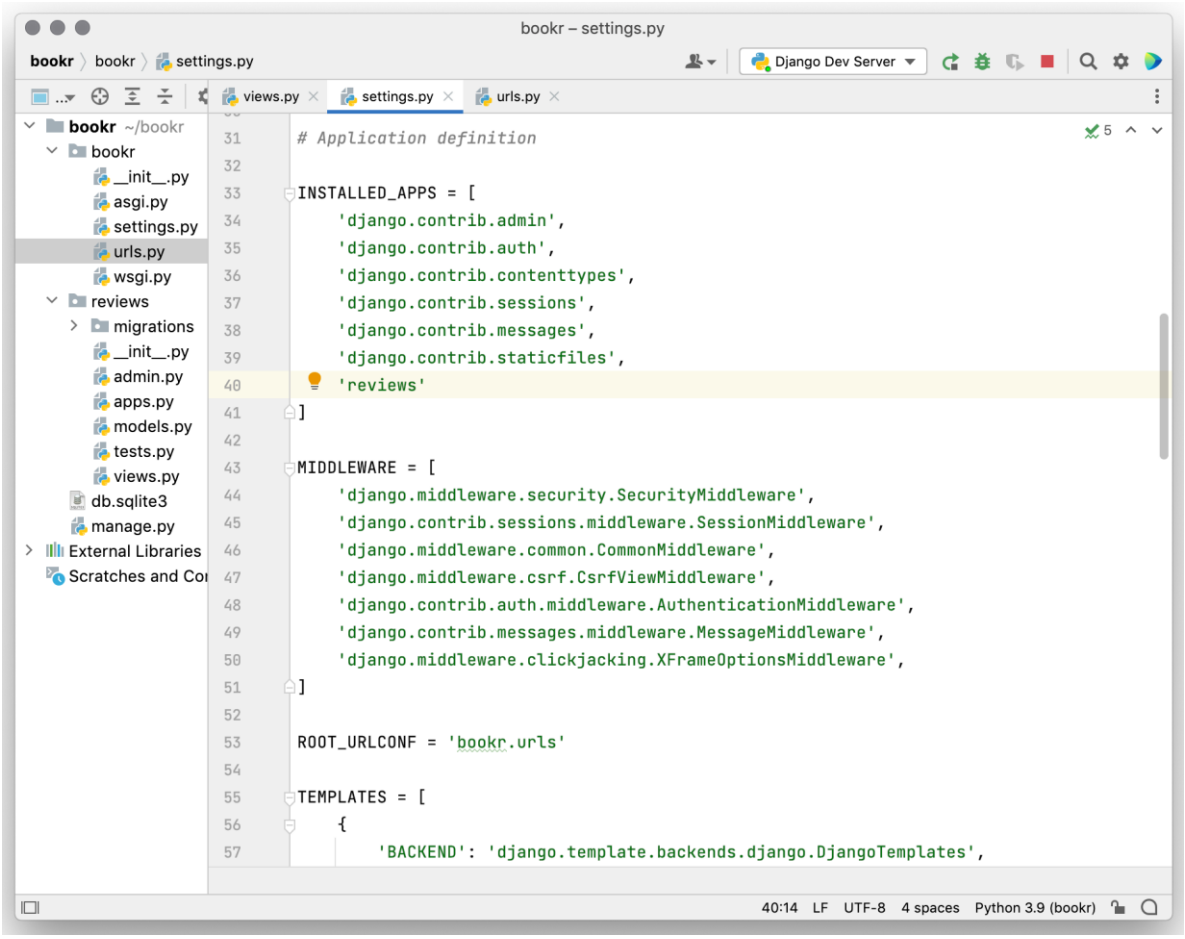
A. Tạo thư mục templates và tập tin template

Bài này sẽ tạo một thư mục templates cho app Reviews sau đó thêm một tập tin template HTML và dùng nó để tạo trang web.

Đầu tiên cần thêm app Reviews vào trong danh sách các app đã được cài đặt (lưu trong thiết đặt INSTALLED_APPS của tập tin settings.py).

1. Mở tập tin **settings.py** trong PyCharm. Thêm mục **reviews** ở cuối của mục thiết đặt INSTALLED_APP. Nội dung phần này sẽ như sau:

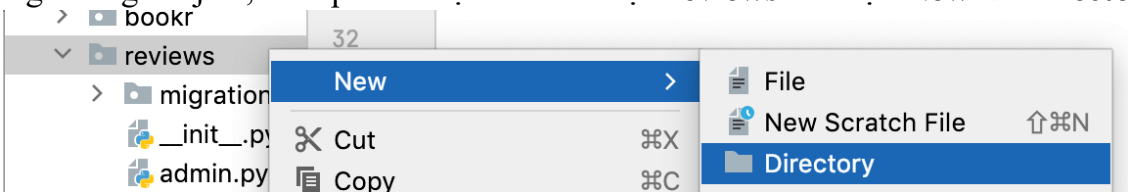
```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'reviews'
]
```



Hình V-1. Nội dung tập tin settings.py sau khi thêm app Review

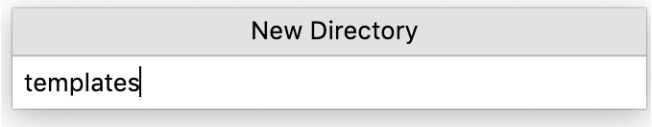
2. Lưu và đóng tập tin **settings.py** lại.

3. Trong khung Project, bấm phải chuột vào thư mục **reviews** và chọn **New → Directory**.



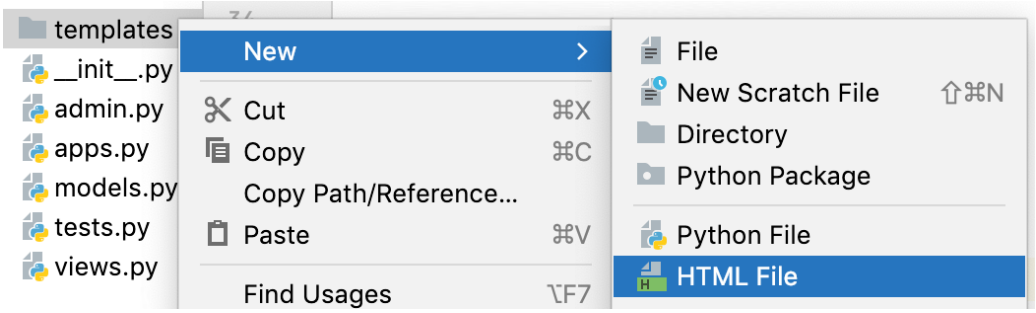
Hình V-2. Chọn mục tạo mới thư mục con trong thư mục reviews

4. Nhập tên thư mục là **templates** và nhấn phím Enter để tạo thư mục.



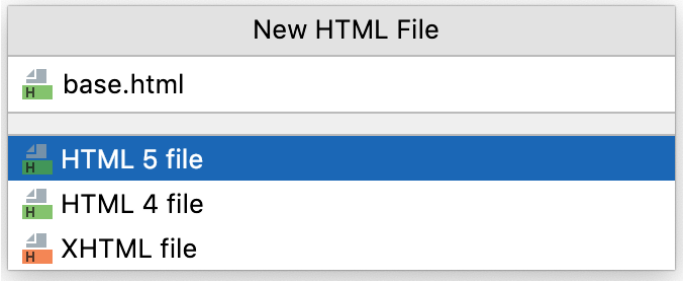
Hình V-3. Nhập tên cho thư mục cần tạo mới là templates

5. Bấm phải chuột vào thư mục **templates** vừa tạo và chọn mục **New → HTML file**.



Hình V-4. Tạo mới tập tin HTML trong thư mục templates

6. Trong cửa sổ **New HTML File**, nhập tên tập tin là **base.html** và chọn loại tập tin là **HTML 5 file** trong ô loại tập tin ở dưới, sau đó nhấn phím Enter để tạo tập tin.



Hình V-5. Cửa sổ New HTML File

7. Sau khi PyCharm tạo tập tin, nó sẽ tự động mở tập tin ra. Nội dung tập tin như sau:

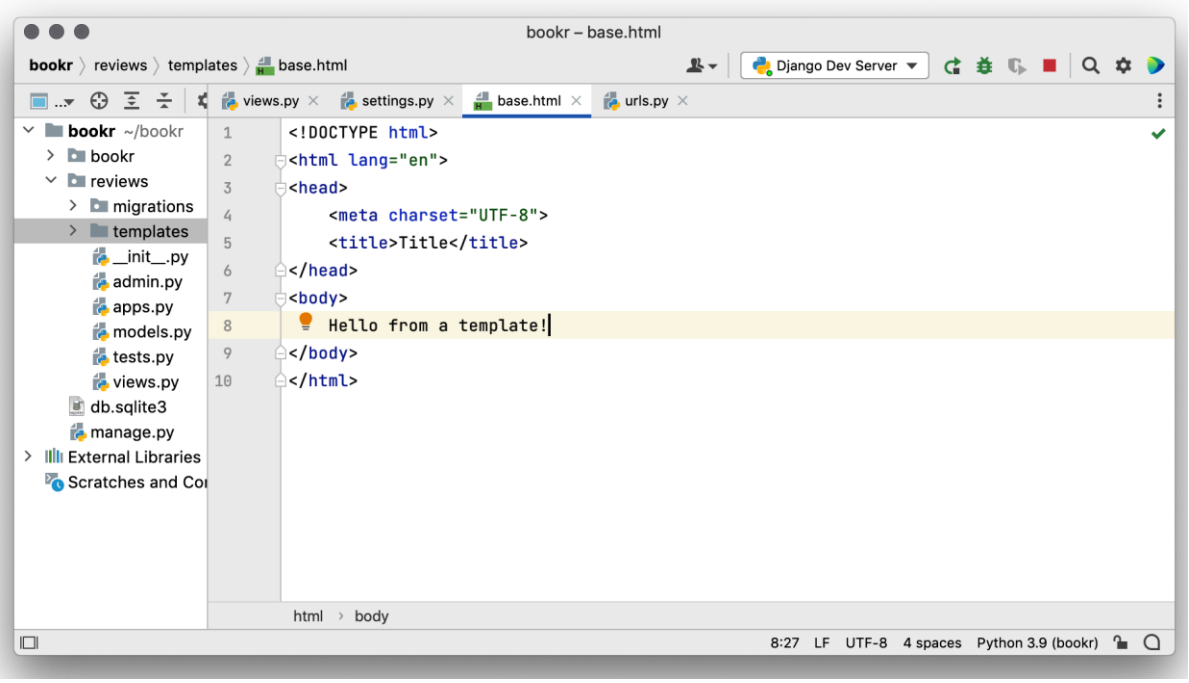
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>

</body>
</html>
```

8. Thêm vào một thông điệp ngắn vào giữa cặp tag **<body>** và **</body>** như sau để kiểm tra Template có được sử dụng không.

```
<body>
    Hello from a template!
</body>
```

Nội dung tập tin sẽ như trong Hình V-6.



Hình V-6. Nội dung tập tin base.html

B. Sử dụng Template trong View

Bài sẽ sửa lại hàm **index** trong tập tin **views.py** để sử dụng template vừa tạo ở phần trên. Ở đây sẽ sử dụng hàm **render** để tải template từ thư mục templates và xuất kết quả sử dụng template này. Phần này sẽ thay thế nội dung văn bản tĩnh xuất ra từ hàm **index**.

1. Trong PyCharm, mở tập tin **views.py** trong thư mục **reviews**.
2. Ở đây chúng ta không còn tạo thông điệp **HttpResponse** thủ công nữa nên xóa dòng **`import HttpRespone`** sau:

```
from django.http import HttpResponse
```

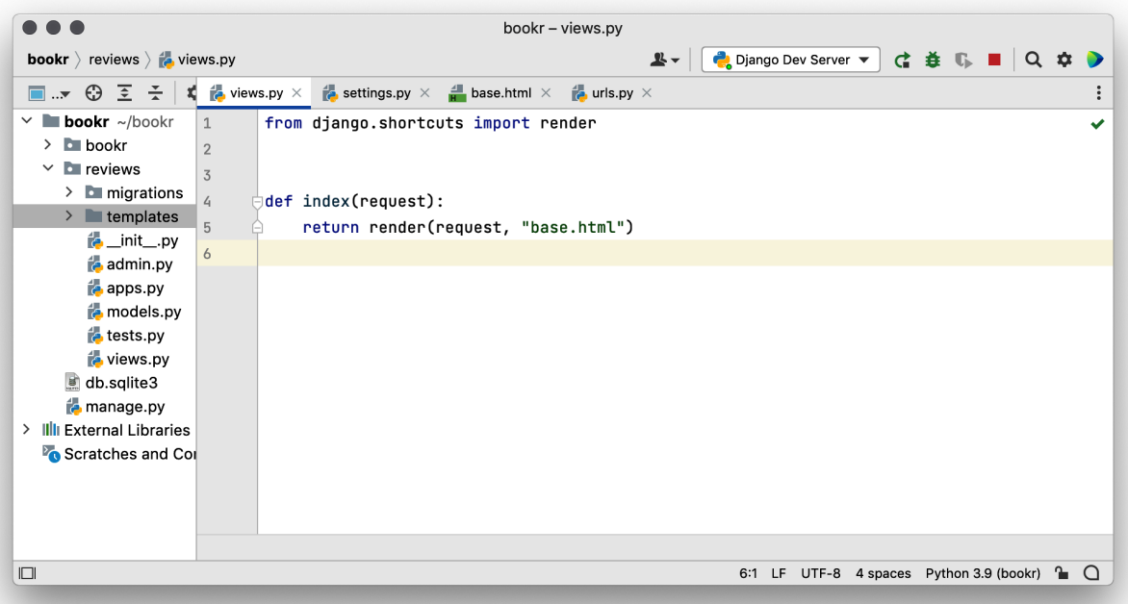
3. Thay thế dòng import trên bằng hàm **render** được import từ gói **django.shortcuts**:

```
from django.shortcuts import render
```

4. Cập nhật hàm **index**: thay vì trả về trực tiếp thông điệp **HttpResponse** ở phần trước, ở đây sẽ trả về kết quả từ lời gọi đến hàm **render** với tham số **request** và **tên của template**:

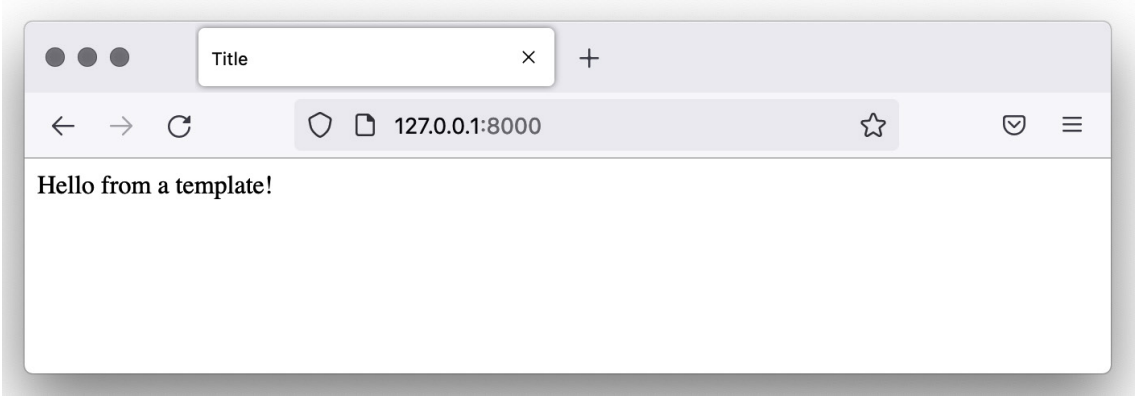
```
def index(request):
    return render(request, "base.html")
```

Nội dung tập tin **views.py** sẽ như Hình V-7.



Hình V-7. Nội dung tập tin view.py khi sử dụng template base.html

5. Khởi chạy máy chủ phát triển (nếu chưa chạy), mở trình duyệt và tải lại trang http://127.0.0.1:8000 . Kết quả sẽ là dòng "**Hello from a template!**" như Hình V-8.



Hình V-8. Trang web sử dụng template

C. Sử dụng các biến trong Template

Bài này sẽ thử sử dụng một biến đơn giản trong tập tin base.html để xuất giá trị lên trang web sử dụng Template.

1. Trong PyCharm, mở tập tin **base.html**.
2. Cập nhật nội dung trong cặp tag **<body>** để chứa một vị trí xuất giá trị của biến **name**:

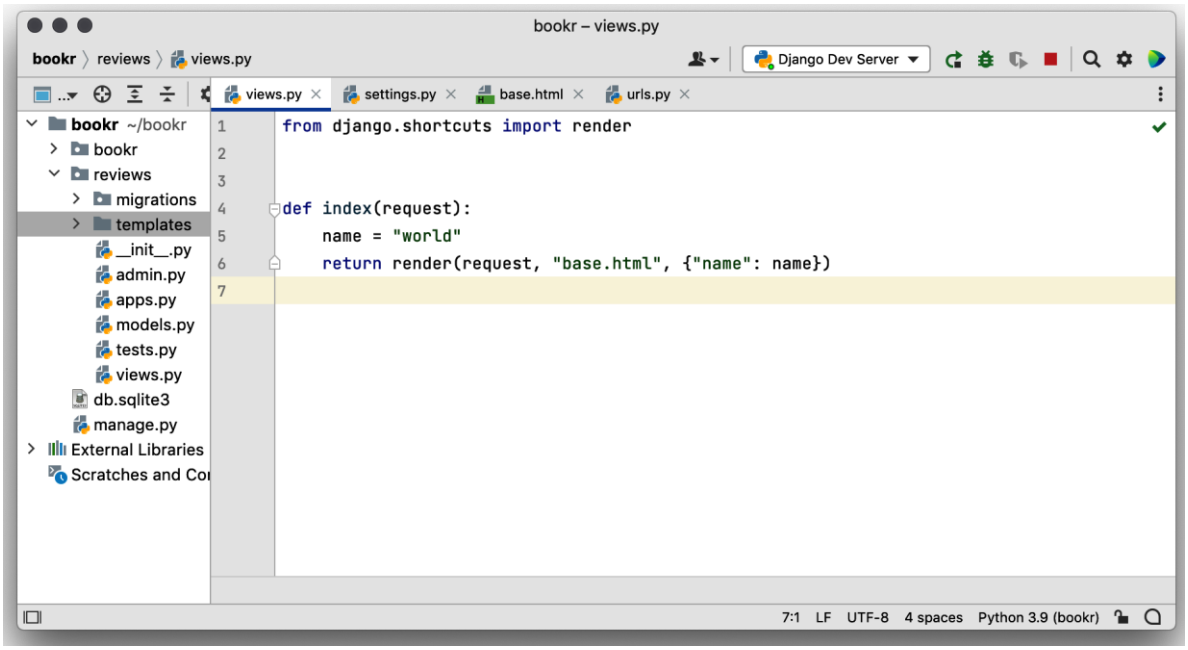
```
<body>
Hello, {{ name }}!
</body>
```

3. Trở lại trình duyệt và và tải lại trang http://127.0.0.1:8000, nội dung trang này sẽ là "**Hello, !**". Đó là vì chúng tac chưa truyền giá trị biến **name** vào hàm **render**.
4. Thực hiện khởi tạo giá trị cho biến **name** và truyền giá trị vào hàm **render** trong phần định nghĩa hàm **index** của tập in **views.py** như sau:

```
def index(request):
```

```
    name = "world"
    return render(request, "base.html", {"name": name})
```

Nội dung tập tin **views.py** sẽ như Hình V-9.



Hình V-9. Nội dung tập tin views.py với biến name

5. Tải lại trang http://127.0.0.1:8000 và kết quả sẽ là "**Hello, world!**".



Hình V-10. Nội dung trang web sau khi truyền biến name qua Template

VI. Quan sát các biến và xử lý lỗi

Khi lập trình, các lập trình viên đều có thể gặp lỗi và cần biết cách xử lý. Khi chương trình có lỗi sẽ xuất hiện thông báo lỗi hoặc trả về kết quả không đúng. Nếu chương trình chỉ trả về kết quả không đúng mà không xuất hiện bất cứ thông báo lỗi nào thì có thể sử dụng tính năng Debug trên PyCharm để tìm hiểu nguyên nhân.

Một số lỗi trên Python và cách thức xử lý lỗi có tại trang 52-53 của Giáo trình.

Sau đây chúng ta sẽ thực hành về cách xử lý lỗi cơ bản và sử dụng tính năng **Debug** trên PyCharm.

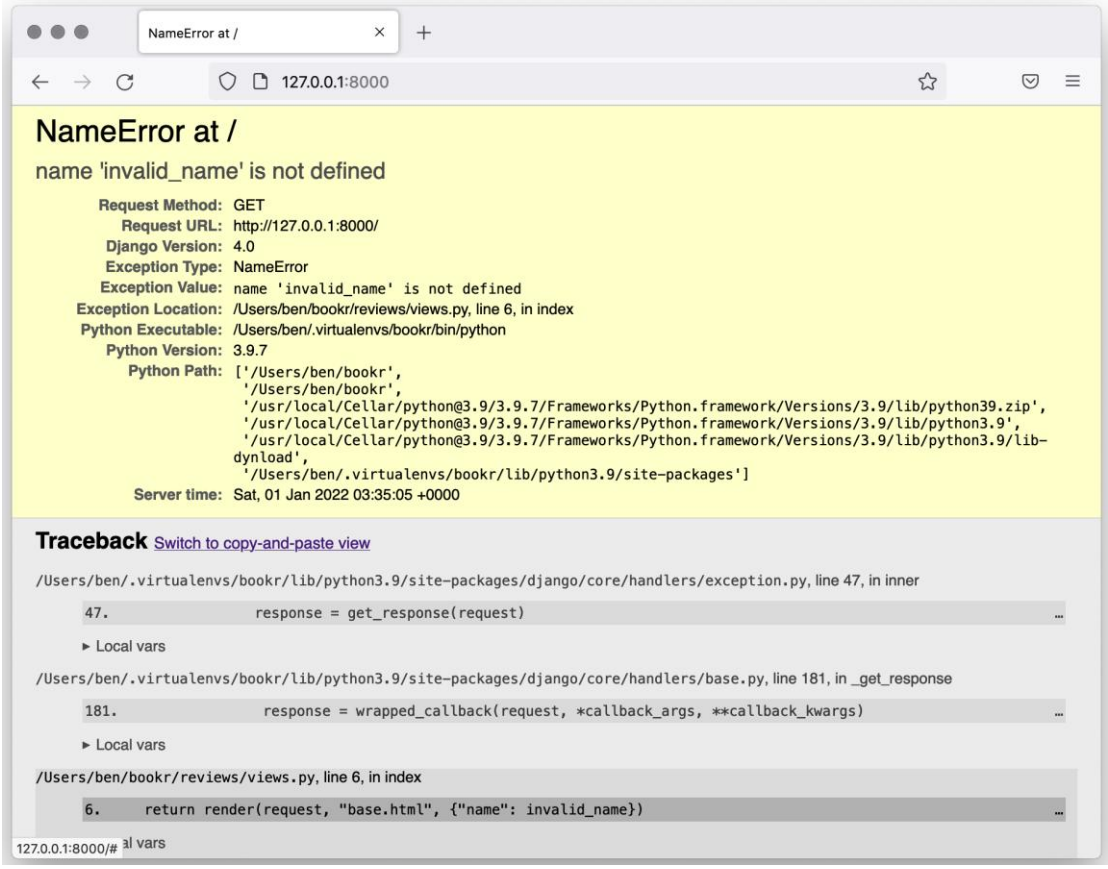
A. Tạo và xem các lỗi

Chúng ta sẽ thử tạo một lỗi và xem nội dung thông báo lỗi trả về. Trong trường hợp này chúng ta sẽ thử sử dụng một biến không tồn tại/ chưa được khai báo và xem lỗi **NameError** hiển thị thế nào.

1. Trong PyCharm, mở tập tin **views.py**. Trong hàm **index**, thay đổi biến truyền vào là **invalid_name**, đây là biến không tồn tại. Hàm index sẽ được sửa đổi thành như sau:

```
def index(request):
    name = "world"
    return render(request, "base.html", {"name": invalid_name})
```

2. Trở lại trình duyệt và tải lại trang http://127.0.0.1:8000. Kết quả sẽ như Hình VI-1.



Hình VI-1. Trang thông báo lỗi của Django

Hai dòng đầu tiên trong phần tiêu đề của trang thông báo lỗi chỉ ra loại lỗi:

```
NameError at /
name 'invalid_name' is not defined
```

3. Nội dung sau phần **Trackback** cho phép truy ngược vết của thông điệp lỗi. Phần này có thể có nhiều khối để cho phép truy ngược lại đến các đoạn mã liên quan với lỗi.
   Trong mỗi khối, khi bấm chuột vào dòng mã liên quan đến lỗi (*có đánh số dòng ở đầu*) sẽ cho phép xem các dòng mã trước và sau dòng này; còn nếu bấm chuột vào **Local vars** sẽ thấy giá trị của các biến liên quan.

   Trong trường hợp này, chúng ta có thể thấy lỗi bắt nguồn từ dòng thứ 6 của tập tin **views.py**, và khi bấm chuột vào mục **Local vars** của khối chứa lỗi này (khối thứ 3), nội dung mở ra sẽ như Hình VI-2. Trong đó, có 2 biến được liệt kê là **name** có giá trị là '**world**' và biến **request** chứa thông tin liên quan đến thông điệp yêu cầu gửi đến từ trình duyệt.



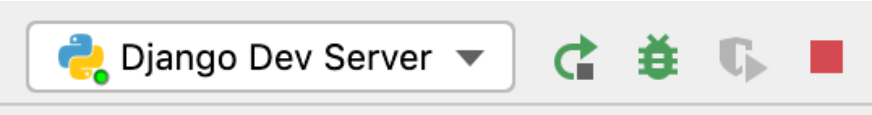
Hình VI-2. Các biến liên quan đến lỗi nằm ở dòng 3 của tập tin views.py

4. Trở lại tập tin **views.py** và sửa lỗi **NameError** trên bằng cách đổi tên biến truyền cho hàm **render** từ **invalid_name** thành **name**.
5. Lưu tập tin và tải lại trang. Kết quả sẽ là trang với nội dung "**Hello, world!**" (như Hình V-10)

B. Tính năng Debug trên Pycharm

Bài này sẽ giới thiệu về tình năng **Debug** của PyCharm. Chúng ta sẽ chạy máy chủ phát triển của Django trong trình gỡ lỗi (debugger) và đặt các ***breakpoint*** *(điểm ngắt)* trong hàm **view** để tạm dừng thực thực thi mã nguồn, từ đó có thể kiểm tra các biến ở từng thời điểm.

1. Nếu máy chủ phát triển web của Django đang chạy thì dừng lại bằng cách nhấn nút **Stop** (hình vuông màu đỏ) trên thanh công cụ phía trên bên phải.

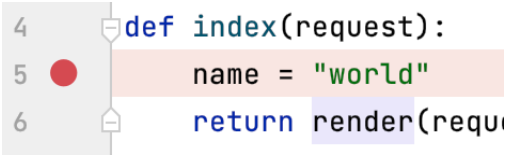


Hình VI-3. Nút Stop và Debug trên thanh công cụ phía trên bên phải của PyCharm

2. Khởi động máy chủ phát triển Django trên trình gỡ lỗi bằng cách nhấn vào nút **Debug** (hình con bọ màu xanh) trên thanh công cụ (cách nút **Stop** 1 nút về phía bên trái).

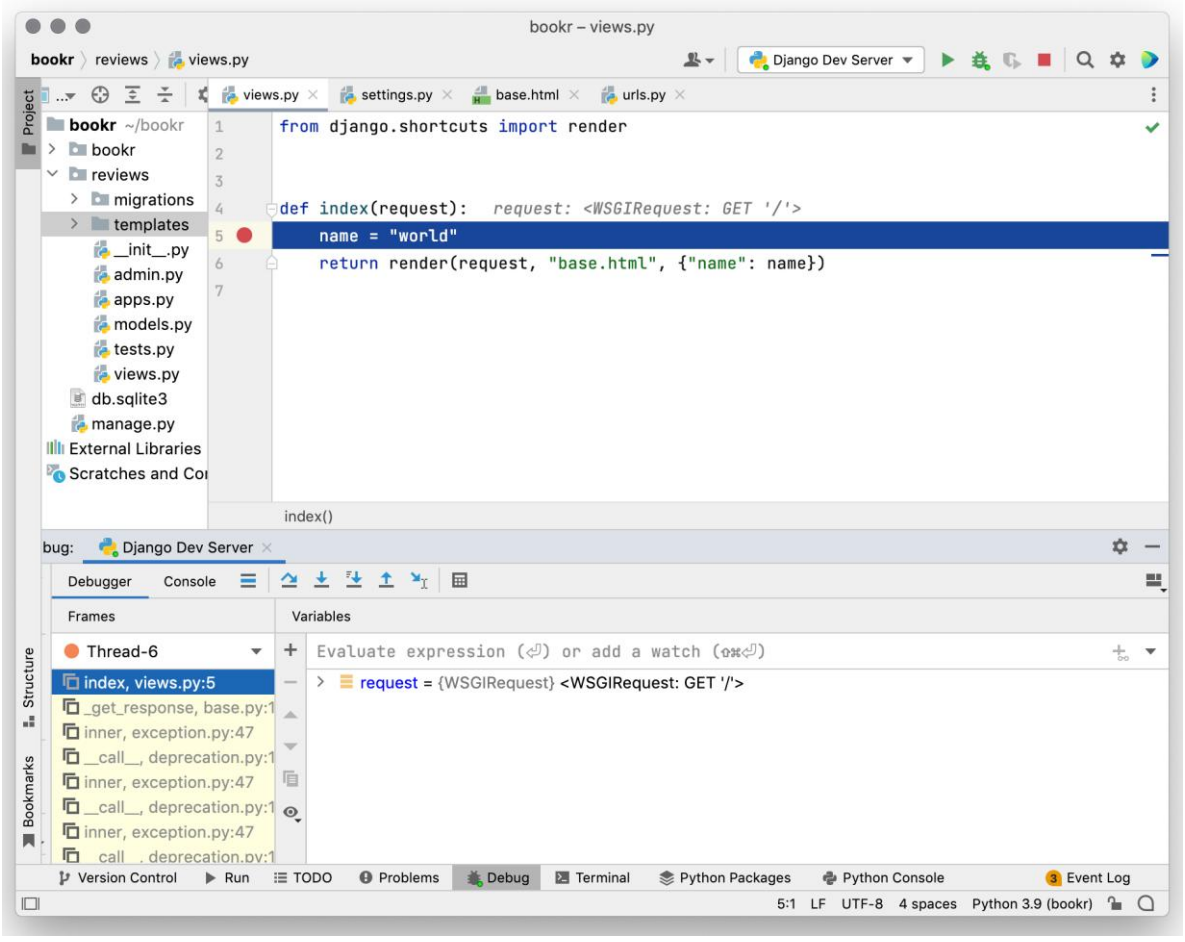
Máy chủ sẽ cần vài giây để khởi động, sau đó chúng ta có thể tải lại trang trên trình duyệt để chắc chắn là trình duyệt vẫn mở trang này như bình thường. Chúng ta chưa thấy có bất cứ thay đổi nào, tất cả các đoạn mã đều đã được thực thi như bình thường.

3. Bây giờ chúng ta đặt một *breakpoint* để dừng thực thi đoạn mã tại vị trí đặt *breakpoint* và xem tình trạng của chương trình. Trong PyCharm, bấm vào phần trống bên phải số thứ tự dòng thứ 5 (nằm ở cạnh trái của khung soạn thảo mã nguồn). Một vòng tròn màu đỏ sẽ xuất hiện để chỉ ra *breakpoint* đã được kích hoạt như trong Hình VI-4.



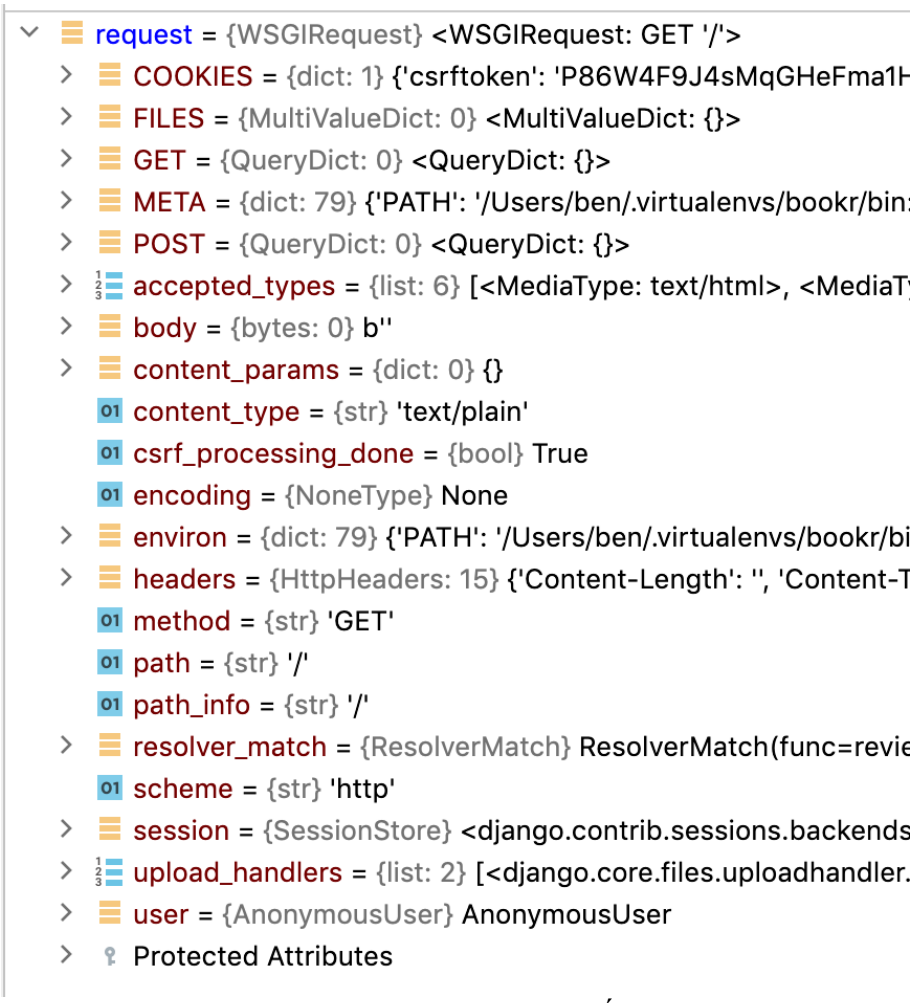
Hình VI-4. Breakpoint tại dòng thứ 5

4. Trở lại trình duyệt và tải lại trang. Trình duyệt sẽ không hiển thị bất cứ nội dung nào mà chỉ trong tình trạng đang tải trang (*loading*). Tùy thuộc vào hệ điều hành mà cần chuyển qua cửa sổ PyCharm hay cửa sổ PyCharm đã tự kích hoạt. Trong cửa sổ PyCharm, chúng ta sẽ thấy dòng thứ 5 đang được làm nổi lên (*highlight*) và khung **Debugger** xuất hiện ở cạnh dưới cửa sổ (xem Hình VI-5). Trong khung **Debugger**, phần bên trái liệt kê chuỗi các hàm đã được gọi để đến dòng mã hiện tại và phần bên phải liệt kê các biến hiện có của hàm.



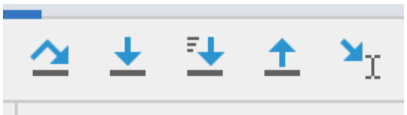
Hình VI-5. Trình gỡ lỗi của PyCharm đã dừng chương trình tại dòng 5

Hiện có 1 biến là **request**. Nếu ta bấm chuột vào phần mũi tên ở trước tên biến sẽ mở ra thông tin chi tiết về các thuộc tính đã được thiết lập. Chúng ta có thể thấy phương thức của Request là '**GET**' (xem Hình VI-6).



Hình VI-6.Các thuộc tính của biến request

5. Phần trên của khung Debugger còn có thanh công cụ **actions** (xem Hình VI-7), với các nút sau:



Hình VI-7. Thanh công cụ actions

+ **Step Over**: Thực thi dòng mã hiện tại và chuyển sang dòng tiếp theo
+ **Step Into**: Thực thi chi tiết từng bước của dòng mã hiện tại. Ví dụ dòng mã này gọi một hàm thì sẽ đi vào thi tiết các dòng mã của hàm được gọi.
+ **Step into My Code**: Thực thi chi tiết từng bước và chỉ dừng lại khi đến đoạn mã chúng ta tự viết.
+ **Step Out**: Quay trở lại hàm hay phương thức đã gọi hàm hiện tại. Ngược với thao tác Step Into.
+ **Run to Cursor**: Nếu ta muốn thực thi một dòng lệnh ở cách xa dòng hiện tại mà không phải bấm chuột vào nút Step Over cho tất cả các dòng nằm giữa chúng, thì tiến hành

như sau: bấm chuột để di chuyển con trỏ văn bản đến dòng lệnh muốn chuyển đến, sau đó bấm vào nút **Run to Cursor**, chương trình sẽ thực thi đến dòng lệnh đó.

Lưu ý: Tùy tình huống để lựa chọn chức năng phù hợp, nhiều chức năng có thể gây bối rối khi sử dụng trong một số trường hợp.

6. Bấm chuột vào nút **Step Over** một lần để thực thi dòng lệnh thứ 5.
7. Ta có thể thấy là biến **name** đã được thêm vào danh sách biến ở phần **Variables** của khung **Debugger** và giá trị của biến là "**world**".



Hình VI-8. Biến name đã được thêm vào danh sách biến ở thời điểm hiện tại

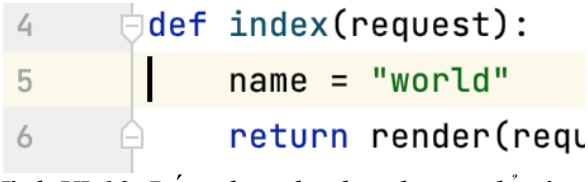
8. Hiện ta đã ở cuối của hàm **index**, nếu ta bấm tiếp vào nút **Step Over** sẽ chuyển đến thực thi mã lệnh ở trong thư viện của Django. Để tiếp tục thực thi và gửi phản hồi về trình duyệt, bấm chuột vào nút **Resume Program** ở cạnh trái của khung **Debugger**. Sau khi bấm, ta sẽ thấy trên trình duyệt đã tải lại xong toàn bộ trang.



Hình VI-9. Các nút để điều khiển việc thực thi chương trình. Nút Resume Program có màu xanh, nằm ở vị trí thứ 3 từ trên xuống

Trên thanh công cụ ở Hình VI-9 còn có các nút khác như: **Rerun** (dừng và chạy debug lại cho máy chủ web), **Modify Run Configuation…** (mở cửa sổ thiết đặt phần run), **Resume Program** (tiếp tục thực thi chương trình cho đến *breakpoint* tiếp theo), **Pause Program** (ngắt chương trình tại điểm thực thi hiện tại), **Stop** (dừng trình gỡ lỗi), **View Breakpoints** (mở một cửa sổ để liệt kê tất cả các *breakpoint* đã đặt) và **Mute Breakpoints** (để bật tắt các *breakpoint* nhưng không xóa đi).

9. Tắt *breakpoint* trong PyCharm bằng cách bấm chuột lên *breakpoint* đó (nút tròn đỏ ở đầu dòng 5)



Hình VI-10. Bấm chuột lên breakpoint để xóa đi